

COMP-421 Database System

School of Computer Science, McGill University

Winter 2025

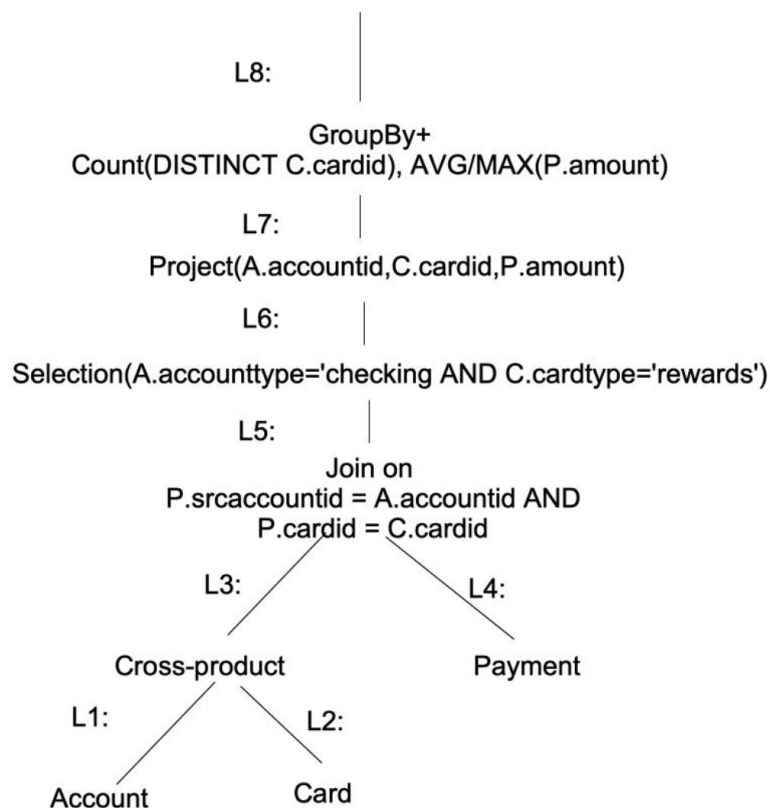
Jocelyne Li (261178604)

Assignment 4: Query Evaluation, Mapy-Reduce and (Transaction Management)

1 Part I: Query Evaluation

1.1 Ex. 1 — Cardinality Estimation and Algebraic Query Optimization (22 points)

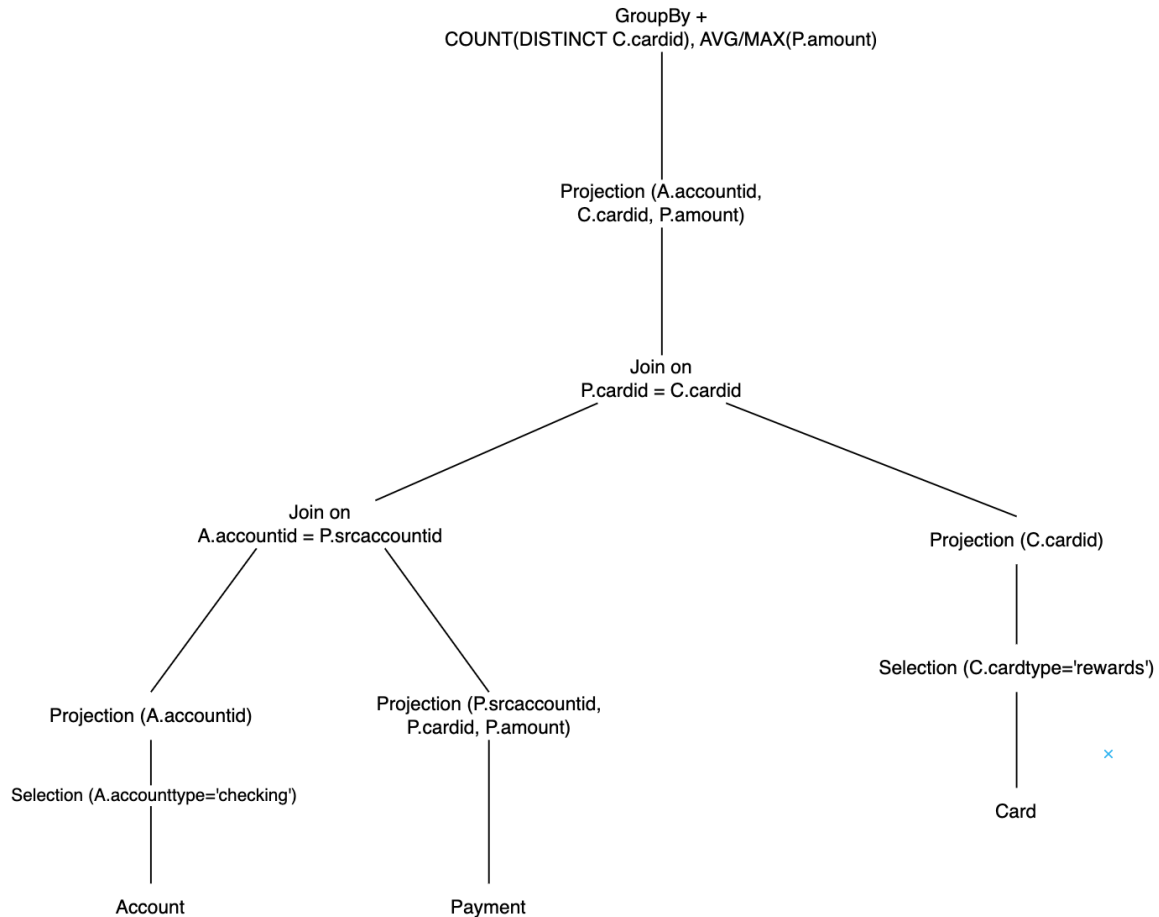
```
SELECT A.accountid, COUNT(DISTINCT C.cardid), AVG(P.amount), MAX(P.amount)
FROM Account A, Card C, Payment P
WHERE P.srcaccountid = A.accountid AND
      P.cardid = C.cardid AND
      A.accounttype='checking' AND
      C.cardtype='rewards'
GROUP BY A.accountid
```



1.(12 points) Indicate the number of tuples and the size of the tuples that flow from the base relations to the first operator and then from one operator to the next. These positions are indicated with L1 to L8.

- **L1 (Account)**
 - Cardinality: 8000
 - Tuple Size: accountid (4B) + name (15B) + email (25B) + address (30B) + balance (4B) + accounttype (6B) = **84B**
- **L2 (Card)**
 - Cardinality: 10,000
 - Tuple Size: cardid (4B) + owneraccountid (4B) + paymentdue (4B) + cardtype (7B) = **19B**
- **L3 (Cross Product: Account $\times$ Card)**
 - Cardinality: $8000 \times 10000 = 80,000,000$
 - Tuple Size: $84B + 19B = \mathbf{103B}$
- **L4 (Payment)**
 - Cardinality: 120,000
 - Tuple Size: paymentid (4B) + srcaccountid (4B) + cardid (4B) + amount (4B) + paydate (10B) = **26B**
- **L5 (Join: P.srcaccountid = A.accountid AND P.cardid = C.cardid)**
 - Cardinality: $\approx 120,000$ (assuming every payment has valid card and account)
 - Tuple Size: $103B + 26B - 4B - 4B = \mathbf{129B}$ (remove duplicate join keys: srcaccountid and cardid)
- **L6 (Selection: A.accounttype='checking' AND C.cardtype='rewards')**
 - Selectivity: $\frac{1}{5} \times \frac{1}{3} = \frac{1}{15}$
 - Cardinality: $120,000 \times \frac{1}{15} = 8000$
 - Tuple Size: **129B**
- **L7 (Projection: A.accountid, C.cardid, P.amount)**
 - Cardinality: 8000
 - Tuple Size: accountid (4B) + cardid (4B) + amount (4B) = **12B**
- **L8 (Group By A.accountid)**
 - Cardinality: $8000 \times \frac{1}{5} = 1600$
 - Tuple Size: accountid (4B) + count (4B) + avg (4B) + max (4B) = **16B**

2.(10 points) Now draw the execution plan after performing an algebraic optimization (pushing down selections, pushing down projections as far as possible, and avoiding cross-product whenever possible. Note that you do not need to indicate how each of the operations would be executed.



3.(0 points) Indicate the number of tuples that flow from the base relations to the first operator and then from one operator to the next for your optimized execution plan.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1.2 Ex. 2 — Cost-based Query Optimization (20 Points)

```
SELECT C.cardtype, COUNT(DISTINCT C.owneraccountid)
FROM Account A, Card C
WHERE A.accountid = C.owneraccountid
      AND A.balance < 6.0
GROUP BY C.cardtype
```

Assumptions:

- Account has 250 pages and 8000 tuples
- Card has 50 pages and 10,000 tuples
- Unclustered B+ tree index on Card.owneraccountid (Type II)
- Uniform distribution of balance over [1, 3000]
- Each qualifying account owns on average $\frac{10,000}{8,000} = 1.25$ cards

Optimization Strategy:

- Push down the selection on Account: `balance < 6.0`
- Use index nested loop join: iterate over qualifying accounts and probe Card using the index on `owneraccountid`
- Group the results by `cardtype` and compute aggregates in memory

Cost Estimation:

1. Selection on Account: `balance < 6.0`

- Selectivity = $\frac{6}{3,000} = \frac{1}{500}$
- Estimated matching tuples: $\frac{8,000}{500} = 16$
- Full scan required (no index on balance): **250 I/Os**

2. Index Lookup in Card:

- For 16 qualifying accounts:
 - Index I/Os: 16 (1 per lookup)
 - Matching cards: $16 \times 1.25 = 20$ (estimated) $\Rightarrow$ Data I/Os (worst case): 20
- Total Card access: $16 + 20 = \mathbf{36 \text{ I/Os}}$

3. Grouping:

- Group by `cardtype` (3 values), in memory: negligible I/O

Total Estimated I/O Cost:

$$250 \text{ (Account scan)} + 16 \text{ (index lookups)} + 20 \text{ (data pages)} = \boxed{286 \text{ I/Os}}$$

1.3 Ex. 3 — Map Reduce (28 points)

Now assume that above relations are much larger. As such, we want to use a map-reduce framework to answer certain queries.

1.(8 points) Output the account ids of those accounts that have two cards. Make sure that you output every account id only once.

Map:

```
For each input record rm of relation Card
    output (rm.owneraccountid, 1)
```

Reduce:

```
For each input record rr = (owneraccountid, list-of-ones)
    if count(list-of-ones) == 2:
        output (owneraccountid)
```

2.(10 points) Indicate how many accounts have 1, 2, 3, etc. cards. The description above indicates that an account has 0, 1 or 2 cards. But create a program here that can handle an arbitrary number of cards. Thus, the output should have tuples (1, x1), (2, x2), (3, x3), etc. where xi is the number of accounts that have i cards. If there is no account that has i cards, then you do not need to output a record for i.

Same map as before, but in reduce, emit a new key representing the count of cards per account. Then a second map reduce counts how many accounts have that number of cards.

First Map:

```
For each input record rm of relation Card
    output (rm.owneraccountid, 1)
```

First Reduce:

```
For each input record rr = (owneraccountid, list-of-ones)
    card_count = count(list-of-ones)
    output (card_count, 1)
```

Second Map:

```
For each input record rm = (card_count, 1)
    output (card_count, 1)
```

Second Reduce:

```
For each input record rr = (card_count, list-of-ones)
    output (card_count, count(list-of-ones))
```

3.(10 points) Adjust the previous query to also output how many accounts have 0 cards.

This time, we're counting the number of accounts overall (including those that do not appear in the Card relation) by mapping over both Account and Card, rather than only accounts that have at least one card.

Map:

```
For each input record rm of relation Card
    output (rm.owneraccountid, "CARD")
```

```
For each input record rm of relation Account
    output (rm.accountid, "ACCOUNT")
```

Reduce:

```
For each input record rr = (accountid, list-of-tags)
    card_count = count occurrences of "CARD" in list-of-tags
    output (card_count, 1)
```

Second Map:

```
For each input record rm = (card_count, 1)
    output (card_count, 1)
```

Second Reduce:

```
For each input record rr = (card_count, list-of-ones)
    output (card_count, count(list-of-ones))
```

1.4 Ex. 4 — Schedules and Concurrency Control (0 Points)

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1.5 Ex. 5 — Schedules and SQL (0 points)

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur

auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.